Software Architecture and Design

Generated: 2026-05-16 17:24

SOFTWARE ARCHITECTURE AND DESIGN

1. ARCHITECTURE STYLE
Recommended architecture: Modular Monolith with Clean Architecture.
This is ideal for a hospital management system at the beginning because it allows strong separation of concerns without the complexity of many microservices.

2. PRIMARY STACK
Frontend:
- React + TypeScript + Vite
- Tailwind CSS
- shadcn/ui
- TanStack Query
- TanStack Table
- Recharts
- SignalR client

Backend:
- ASP.NET Core Web API
- C#
- Clean Architecture
- Entity Framework Core
- Dapper for heavy read/report queries
- SignalR
- Hangfire
- Serilog
- FluentValidation

Database:
- Microsoft SQL Server first
- Oracle-ready design later if needed
- Use object storage / PACS for MRI, CT, X-ray and imaging files

AI (last phase):
- Separate Python FastAPI service
- Azure OpenAI / OpenAI for initial LLM tasks
- Later optional local models
- AI audit trail and human approval flow

3. MAIN LAYERS
Presentation Layer:
- React application
- Role-based dashboards
- Forms, tables, workflows, bed board, reporting UI

API Layer:
- ASP.NET Core controllers/endpoints
- Authentication and authorization
- Validation and API documentation

Application Layer:
- Use cases / services
- Business workflow orchestration
- Admission, billing, pharmacy, lab, nursing, etc.

Domain Layer:
- Core business rules and entities
- Patients, visits, admissions, beds, invoices, prescriptions, lab orders

Infrastructure Layer:
- Database access (EF Core, Dapper)
- Reporting engine
- File storage integration
- Notifications
- Background jobs
- External integrations

4. CORE MODULES
- Identity / Users / Roles / Permissions
- Patient Management
- Appointment / Queue
- OPD
- Emergency
- Admission
- Building / Floor / Room / Bed Management
- Nursing
- Lab
- Radiology
- Pharmacy
- Billing
- Operation Theater
- Inventory
- Reporting
- Audit
- Notifications
- AI (later)

5. DATA AND FILE STORAGE PRINCIPLE
- Transactional data in SQL Server
- Heavy medical images in DICOM/PACS + object storage
- Database stores references, metadata, and report links

6. REPORTING DESIGN
Use a layered reporting approach:
- Operational dashboards inside React
- Structured document PDFs using QuestPDF
- Enterprise tabular / paginated reports via SSRS or FastReport

- Scheduled background report generation via Hangfire
- AI-generated narrative summaries in a later phase only

7. REAL-TIME DESIGN
SignalR channels for:
- Queue updates
- Bed board updates
- New admissions
- Lab order notifications
- Pharmacy requests
- Nursing task alerts
- Billing clearance updates

8. SECURITY DESIGN
- ASP.NET Identity + JWT / refresh token
- Role-based and permission-based access control
- Audit logging for all critical actions
- Encrypted transport (HTTPS)
- Backup and disaster recovery strategy
- Separation of production / test environments

9. DEPLOYMENT DESIGN
Development:
- Docker Compose
- SQL Server
- Redis
- MinIO / object storage
- API
- Frontend

Production:
- Docker containers
- Reverse proxy
- Managed or hardened SQL Server
- Redis
- Object storage
- Monitoring/logging stack

Later Enterprise:
- Kubernetes
- Horizontal scaling
- CI/CD pipelines
- Centralized observability

10. AI INTEGRATION DESIGN
AI is added after the core HMS is stable.
Flow:
User action → ASP.NET backend verifies permissions → relevant data prepared →
minimal necessary data sent to Python AI service → AI draft returned →
human review required → final save in HMS with audit trail.

11. WHY THIS ARCHITECTURE
- Good for enterprise workflow
- Easier to build with Codex in phases
- Keeps AI isolated and safer
- Scales from MVP to enterprise deployment
- Works well with hospital operational complexity

Architecture Workflow Diagram